TESTE DE CAIXA BRANCA — SISTEMA DE PEDIDOS — LOJA SENAI

Instituição: SESI-SENAI

Curso: Desenvolvimento de Sistemas

Unidade Curricular: Testes de Software

Atividade: Teste de Caixa Branca — Sistema de Pedidos

Aluno: Gabrielly Piffer

Turma: 2DES_B

Professor: Robson e Reenye

Data: 30/09/2026

1. CONTEXTUALIZAÇÃO SOBRE TESTE DE CAIXA BRANCA

O teste de caixa branca é uma técnica de teste de software que considera a estrutura interna do código para verificar seu funcionamento.

Diferentemente de um teste que analisa somente as entradas e saídas, o teste de caixa branca permite observar os comandos, condições, decisões e diferentes caminhos que podem ser executados durante o funcionamento do programa.

Nesta atividade foi analisado um sistema de pedidos desenvolvido em HTML, CSS e JavaScript.

O sistema permite que o usuário:

- escolha um produto;
- informe a quantidade;
- utilize um cupom de desconto;
- escolha o tipo de frete;
- calcule o valor final do pedido.

O sistema possui três produtos:

Produto	Preço	Estoque
Notebook	R$ 3.000,00	5
Mouse	R$ 80,00	20
Teclado	R$ 150,00	10

Também existem três opções de frete:

- Frete normal: R$ 30,00;
- Frete expresso: R$ 60,00;
- Retirada na loja: grátis.

O sistema possui regras de desconto:

- **SENAI10:** 10% de desconto;
- **SENAI20:** 20% de desconto para compras a partir de R$ 1.000,00;
- **Desconto por quantidade:** 5% para compras com mais de 5 unidades;
- **Desconto para pedido de alto valor:** 5% quando o total ultrapassa R$ 3.000,00.

O sistema também possui a regra de **frete normal grátis para compras a partir de R$ 500,00**.

Essas regras fazem parte do funcionamento do sistema. Portanto, quando uma regra for aplicada, o sistema deve informar claramente ao cliente o motivo e o valor do benefício.

O objetivo desta atividade é analisar o código fornecido, identificar os seis problemas de lógica e comunicação encontrados, demonstrar os caminhos percorridos pelo programa e realizar as correções necessárias.

2. ANÁLISE DAS ESTRUTURAS DE DECISÃO

Durante a análise do código foram encontradas diversas estruturas condicionais que alteram o caminho de execução do sistema.

2.1 Validação da quantidade

O código original verifica a quantidade utilizando:

```
if (qtd < 0) {
   resultado.innerHTML = "<p>Quantidade inválida.</p>";
```

```
    return;
}
```

O problema está na condição utilizada.

A quantidade `0` não é válida para um pedido, porém `0 < 0` resulta em falso.

Por isso, o sistema continua o processamento.

A correção realizada foi:

```
if (qtd < 1) {
```

Dessa forma, qualquer quantidade menor que 1 será considerada inválida.

2.2 Verificação do estoque

O código original utiliza:

```
if (qtd >= estoque[produtoSelecionado]) {
    resultado.innerHTML = "<p>Quantidade indisponível em
estoque.</p>";
    return;
}
```

O operador >= faz com que uma quantidade exatamente igual ao estoque seja bloqueada.

Por exemplo, se existem 5 notebooks disponíveis e o cliente solicita exatamente 5, o sistema considera a quantidade indisponível.

A correção realizada foi:

```
if (qtd > estoque[produtoSelecionado]) {
```

Assim, a compra é bloqueada somente quando a quantidade solicitada for maior que o estoque disponível.

2.3 Cálculo dos descontos

A função de desconto possui as seguintes regras:

```
function calcularDesconto(subtotal, codigo) {
   if (codigo === "SENAI10") {
       return subtotal * 0.10;
   }

   if (codigo === "SENAI20" && subtotal >= 1000) {
       return subtotal * 0.20;
   }

   return 0;
}
```

O cupom `SENAI10` concede 10% de desconto.

O cupom `SENAI20` concede 20% de desconto quando o subtotal é igual ou superior a R$ 1.000,00.

Essas condições fazem parte das regras do sistema e **não são consideradas erros**.

O problema identificado está na comunicação do resultado.

Quando um cupom é aplicado, o sistema deve informar ao cliente:

- qual cupom foi utilizado;
- qual percentual foi aplicado;
- quanto foi descontado.

Quando o `SENAI20` não atingir o valor mínimo, o sistema também deve informar que o desconto não foi aplicado e explicar que é necessário atingir R$ 1.000,00.

2.4 Cálculo do frete

O código original possui:

```
if (tipo === "retirada") {
   return 0;
}

if (tipo === "expresso") {
   return 60;
}
```

```
if (subtotal >= 500) {
   return 0;
}

return 30;
```

Durante a análise, foi verificado que a condição:

```
if (subtotal >= 500) {
   return 0;
}
```

representa uma regra de negócio do sistema: **frete normal grátis a partir de R$ 500,00**.

Portanto, essa condição não deve ser removida.

O problema identificado é a falta de comunicação clara dessa regra ao cliente.

Quando o frete normal ficar grátis, o sistema deverá informar que isso ocorreu porque a compra atingiu o valor mínimo de R$ 500,00.

Quando o valor for inferior a R$ 500,00, o sistema também deverá informar que o frete normal custa R$ 30,00 e qual valor falta para atingir o frete grátis.

2.5 Desconto por quantidade

O código possui:

```
if (qtd > 5) {
   total = total - subtotal * 0.05;
}
```

Essa condição representa uma regra de desconto por quantidade.

Quando o cliente compra mais de 5 unidades, é aplicado um desconto de 5%.

O desconto não deve ser removido.

O problema identificado é que o cliente não era informado claramente de que o desconto havia sido aplicado.

A correção consiste em manter o cálculo e adicionar uma mensagem explicando:

- que o desconto foi aplicado;
- que ele ocorre porque foram compradas mais de 5 unidades;
- qual foi o valor descontado.

2.6 Desconto para pedidos acima de R$ 3.000,00

O código possui:

```
if (total > 3000) {
   total = total * 0.95;
}
```

Essa condição aplica 5% de desconto quando o valor do pedido ultrapassa R$ 3.000,00.

Assim como o desconto por quantidade, essa regra deve ser mantida.

O problema identificado está na comunicação com o cliente.

Quando o desconto for aplicado, o sistema deverá informar que o pedido ultrapassou R$ 3.000,00 e que foi concedido um desconto adicional de 5%.

2.7 Classificação do pedido

O sistema também possui:

```
if (total <= 0) {
   mensagem = "Valor do pedido inválido.";
} else if (total >= 3000) {
   mensagem = "Pedido de alto valor.";
}
```

Essa estrutura classifica o pedido depois do cálculo do total.

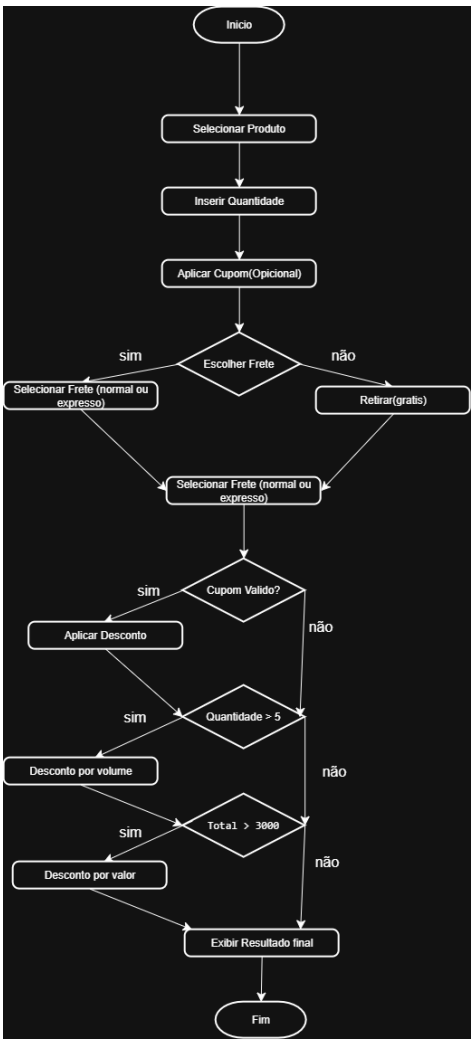
Os caminhos possíveis são:

- Total menor ou igual a zero → valor inválido;
- Total maior que zero e menor que R$ 3.000,00 → pedido calculado com sucesso;
- Total igual ou superior a R$ 3.000,00 → pedido de alto valor.

Essa estrutura foi mantida.

3. FLUXOGRAMA DO EXEMPLO

O fluxo geral do sistema pode ser representado da seguinte forma:



Os seis problemas foram analisados individualmente através dos caminhos condicionais apresentados no código.

4. CASOS DE TESTE — SEIS PROBLEMAS ENCONTRADOS

ERRO 1 — Quantidade igual a zero

Nível: facil

Trecho do código

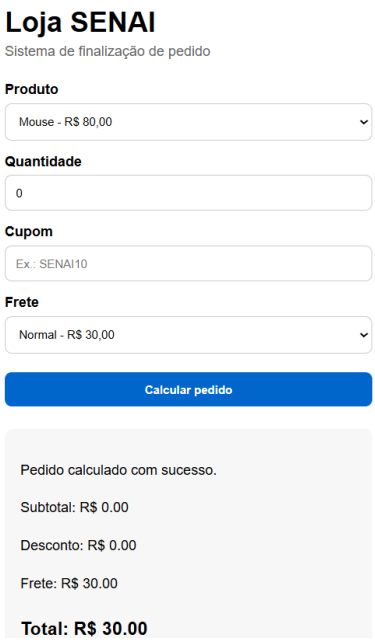
```
if (qtd < 0) {
   resultado.innerHTML = "<p>Quantidade inválida.</p>";
   return;
}
```

Comportamento esperado

O sistema deve impedir qualquer quantidade menor que 1.

Dados utilizados no teste

- Produto: Mouse
- Quantidade: 0
- Cupom: nenhum
- Frete: normal



Resultado esperado

O sistema deveria informar:

```
Quantidade inválida.
A quantidade deve ser maior que zero.
```

Resultado obtido

O sistema aceitava a quantidade 0 e continuava o processamento.

Erro identificado

A condição utilizada não bloqueava o valor zero.

Correção realizada

Alterar:

```
if (qtd < 0)
```

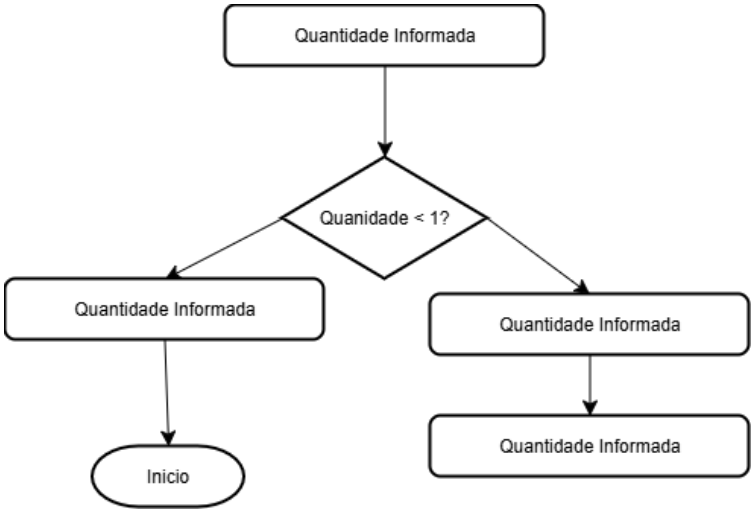
para:

```
if (qtd < 1)
```

Resultado após a correção

Com quantidade 0, o sistema bloqueia o pedido e apresenta a mensagem de quantidade inválida.

Fluxograma



ERRO 2 — Quantidade igual ao estoque

Nível: Fácil

Trecho do código

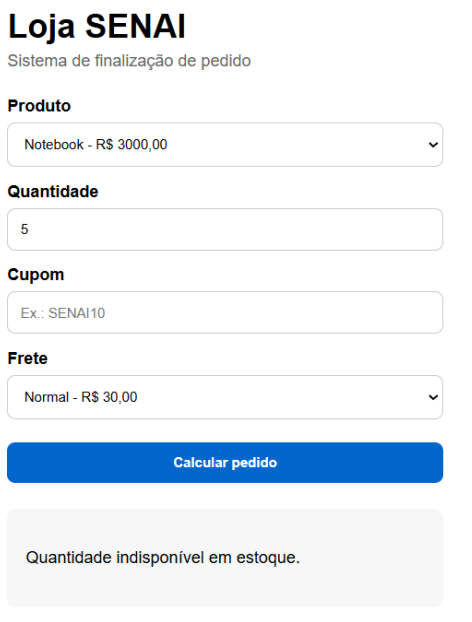
```
if (qtd >= estoque[produtoSelecionado]) {
   resultado.innerHTML =
   "<p>Quantidade indisponível em estoque.</p>";
   return;
}
```

Comportamento esperado

O sistema deve permitir a compra exatamente na quantidade disponível no estoque.

Dados utilizados no teste

- Produto: Notebook
- Estoque: 5 unidades
- Quantidade: 5
- Cupom: nenhum
- Frete: normal



Resultado esperado

A compra de 5 notebooks deveria ser permitida.

Resultado obtido

O sistema informa:

`Quantidade indisponível em estoque.`

Erro identificado

O operador >= também considera como inválida a quantidade exatamente igual ao estoque.

Correção realizada

Alterar:

```
if (qtd >= estoque[produtoSelecionado])
```

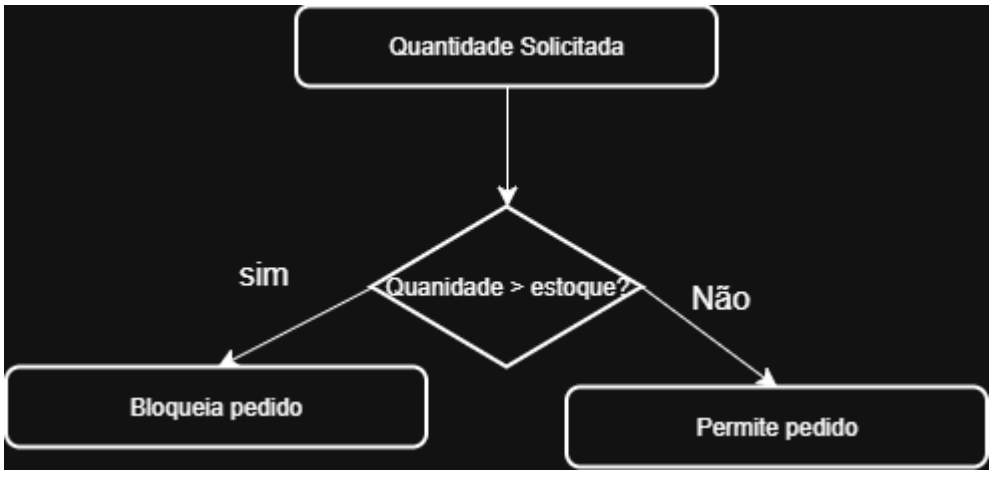
para:

```
if (qtd > estoque[produtoSelecionado])
```

Resultado após a correção

A compra de exatamente 5 notebooks passa a ser permitida.

Fluxograma



ERRO 3 — Falta de comunicação sobre o frete grátis

Nível: Medio

Trecho do código

```
if (subtotal >= 500) {
   return 0;
}

return 30;
```

Comportamento esperado

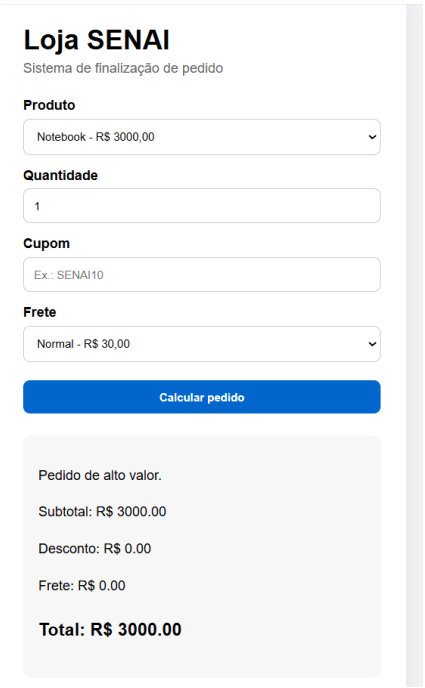
Quando o cliente escolher o frete normal:

- abaixo de R$ 500,00 → frete de R$ 30,00;
- a partir de R$ 500,00 → frete grátis.

O sistema deve informar ao cliente quando o frete grátis for aplicado.

Dados utilizados no teste

- Produto: Notebook
- Quantidade: 1
- Subtotal: R$ 3.000,00
- Cupom: nenhum
- Frete: normal



Resultado esperado

O sistema deve apresentar:

```
Frete normal grátis!
Sua compra atingiu R$ 500,00 ou mais.
Economia no frete: R$ 30,00.
```

Resultado obtido

O sistema apresentava somente:

```
Frete: R$ 0,00
```

O cliente não recebia a informação de que o frete estava grátis devido ao valor da compra.

Erro identificado

A regra de frete grátis funcionava, porém o resultado não explicava ao cliente **por que o frete foi zerado**.

Correção realizada

A lógica do frete foi mantida:

```
if (subtotal >= 500) {
   return 0;
}
```

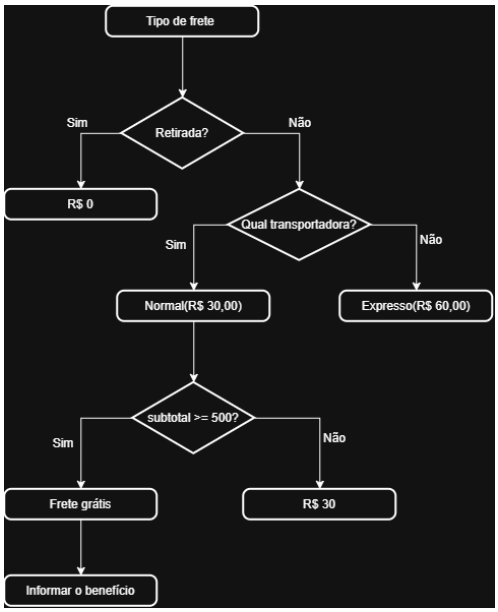
Foi adicionada uma descrição para o cliente:

```
if (tipoFrete === "normal" && subtotal >= 500) {
   descricaoFrete = `
      <p><strong>Frete normal grátis!</strong></p>
      <p>Sua compra atingiu R$ 500,00 ou mais,
      por isso o frete normal não será cobrado.</p>
      <p>Economia no frete: R$ 30,00</p>
   `;
}
```

Resultado após a correção

O sistema continua aplicando o frete grátis, mas agora informa claramente o benefício ao cliente.

Fluxograma



ERRO 4 — Falta de comunicação sobre o desconto por quantidade

Nível: Medio

Trecho do código

```
if (qtd > 5) {
   total = total - subtotal * 0.05;
}
```

Comportamento esperado

Quando o cliente comprar mais de 5 unidades, o sistema deve aplicar o desconto de 5% e informar claramente ao cliente.

Dados utilizados no teste

- Produto: Mouse
- Quantidade: 7
- Preço unitário: R$ 80,00
- Subtotal: R$ 560,00
- Cupom: nenhum
- Frete: normal



Resultado esperado

O sistema deve aplicar:

```
R$ 560,00 × 5% = R$ 28,00
```

E informar:

```
Desconto por quantidade aplicado!
Como foram compradas mais de 5 unidades,
você recebeu 5% de desconto.

Desconto: - R$ 28,00
```

Resultado obtido

O desconto era aplicado no cálculo, mas o sistema não informava claramente ao cliente que o desconto ocorreu por causa da quantidade.

Erro identificado

A regra de desconto funcionava, porém faltava comunicação sobre sua aplicação.

Correção realizada

A regra foi mantida:

```
if (qtd > 5) {
   descontoQuantidade = subtotal * 0.05;
```

```
   total = total - descontoQuantidade;
}
```

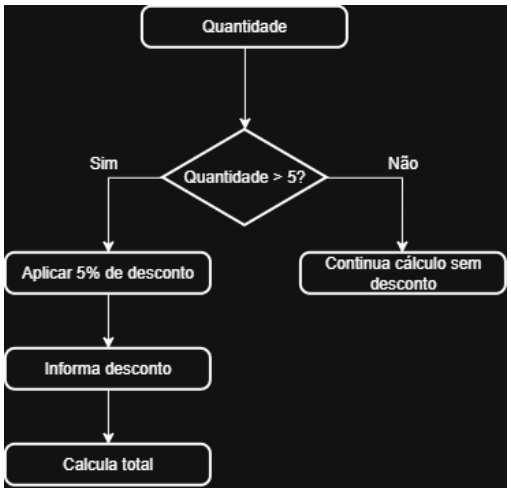
Foi adicionada uma mensagem:

```
<p><strong>Desconto por quantidade aplicado!</strong></p>
<p>Como foram compradas mais de 5 unidades,
você recebeu 5% de desconto.</p>
<p>Desconto: - R$ ${descontoQuantidade.toFixed(2)}</p>
```

Resultado após a correção

O sistema continua aplicando os 5% e agora informa ao cliente o motivo e o valor do desconto.

Fluxograma



ERRO 5 — Falta de comunicação sobre o desconto de alto valor

Nível: Dificil

Trecho do código

```
if (total > 3000) {
   total = total * 0.95;
}
```

Comportamento esperado

Quando o pedido ultrapassar R$ 3.000,00, o desconto de 5% deve ser aplicado e identificado ao cliente.

Dados utilizados no teste

- Produto: Notebook
- Quantidade: 1
- Subtotal: R$ 3.000,00
- Cupom: nenhum
- Frete: expresso
- Frete: R$ 60,00



Resultado esperado

O sistema deve informar:

```
Desconto de pedido de alto valor aplicado!

Como o valor do pedido ultrapassou R$ 3.000,00,
foi aplicado mais 5% de desconto.

Desconto: - R$ 153,00
```

Resultado obtido

O desconto era aplicado ao total, porém o sistema não informava claramente ao cliente a origem do desconto.

Erro identificado

A regra era executada, mas não era apresentada de forma clara no resultado.

Correção realizada

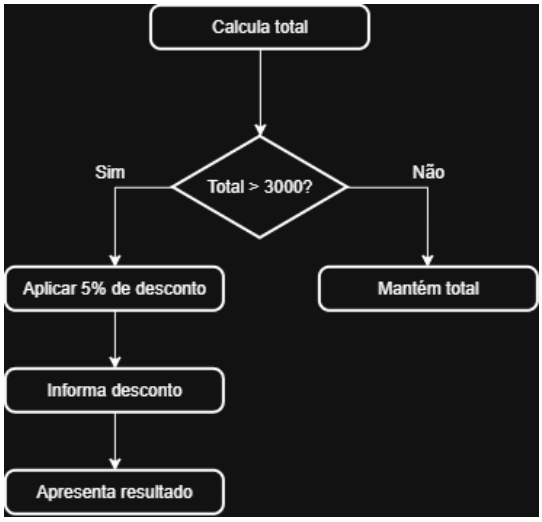
A regra foi mantida e foi criada uma descrição:

```
if (total > 3000) {
   descontoAltoValor = total * 0.05;
   total = total - descontoAltoValor;
}
```

Resultado após a correção

O sistema mantém o desconto de 5% e informa ao cliente o motivo e o valor aplicado.

Fluxograma



ERRO 6 — Falta de comunicação sobre a combinação dos descontos

Nível: Dificil

Trecho do código relacionado

```
let total = subtotal - desconto + valorFrete;

if (qtd > 5) {
   total = total - subtotal * 0.05;
```

```
}

if (total > 3000) {
   total = total * 0.95;
}
```

Comportamento esperado

O sistema deve calcular corretamente todas as regras aplicáveis e informar ao cliente cada desconto que foi utilizado.

O cliente deve conseguir identificar:

- desconto do cupom;
- desconto por quantidade;
- desconto por pedido de alto valor;
- frete;
- total final.

Dados utilizados no teste

- Produto: Notebook
- Quantidade: 2
- Preço unitário: R$ 3.000,00
- Subtotal: R$ 6.000,00
- Cupom: SENAI10
- Frete: normal



Resultado esperado

O sistema deve apresentar claramente:

```
Subtotal: R$ 6.000,00

Cupom SENAI10 aplicado.
Desconto de 10%: - R$ 600,00

Frete normal grátis.
Sua compra atingiu R$ 500,00 ou mais.
Economia no frete: R$ 30,00.

Desconto de pedido de alto valor aplicado.
Desconto de 5%: - R$ 270,00

Total: R$ 5.130,00
```

Resultado obtido

O sistema realizava os cálculos, porém o resultado não identificava claramente todos os benefícios aplicados.

Erro identificado

O problema estava principalmente na comunicação do resultado. O cliente não conseguia identificar facilmente quais condições foram executadas e quais descontos foram aplicados.

Correção realizada

Foram criadas variáveis específicas para identificar os descontos:

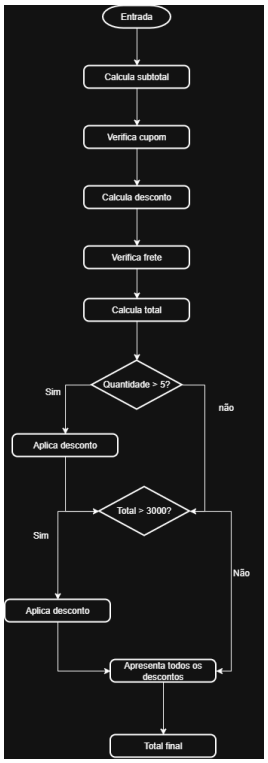
```
let descontoQuantidade = 0;
let descontoAltoValor = 0;
```

Também foram adicionadas mensagens específicas para cada regra.

Resultado após a correção

O sistema apresenta separadamente cada desconto e o valor correspondente, permitindo que o cliente compreenda como o total foi calculado.

Fluxograma



5. CASOS DE TESTE

Os casos de teste foram definidos para cobrir diferentes caminhos das estruturas condicionais encontradas no código.

Identificação	Entrada	Condição/Caminho	Resultado Esperado
CT01	Mouse, quantidade 0	`qtd < 1`	Quantidade inválida
CT02	Notebook, quantidade 5	`qtd > estoque`	Pedido permitido
CT03	Notebook, quantidade 1, frete normal	Subtotal ≥ R$ 500	Frete grátis identificado
CT04	Mouse, quantidade 7	`qtd > 5`	Desconto de 5% identificado
CT05	Notebook, quantidade 1, frete expresso	Total > R$ 3.000	Desconto de 5% identificado
CT06	Notebook, quantidade 2, SENAI10, frete normal	Combinação de regras	Todos os descontos e benefícios identificados

6. RESULTADOS DOS TESTES

Teste	Entrada	Resultado Esperado	Resultado Obtido	Situação
CT01	Mouse / 0	Quantidade inválida	Quantidade era aceita	Corrigido
CT02	Notebook / 5	Pedido permitido	Pedido era bloqueado	Corrigido
CT03	Notebook / 1 / Normal	Frete grátis informado	Frete aparecia apenas como R$ 0,00	Corrigido
CT04	Mouse / 7	Desconto de 5% informado	Desconto era aplicado sem explicação	Corrigido
CT05	Notebook / 1 / Expresso	Desconto de alto valor informado	Desconto era aplicado sem explicação	Corrigido
CT06	Notebook / 2 / SENAI10 / Normal	Todos os benefícios identificados	Cálculos eram realizados sem detalhamento suficiente	Corrigido

7. TESTE DAS REGRAS DE DESCONTO E FRETE

Além dos testes dos seis problemas encontrados, foram realizados testes específicos para verificar as regras corretas do sistema.

CT07 — SENAI10

Entrada

- Produto: Notebook
- Quantidade: 1
- Cupom: SENAI10
- Frete: normal

Cálculo

Subtotal:

`R$ 3.000,00`

Desconto de 10%:

`R$ 3.000,00 × 10% = R$ 300,00`

Frete normal:

```
R$ 0,00
```

Como o subtotal é maior que R$ 500, o frete normal é grátis.

Total:

```
R$ 3.000,00 - R$ 300,00 + R$ 0,00
= R$ 2.700,00
```

Resultado esperado

O sistema deve identificar:

```
Cupom SENAI10 aplicado!
Você recebeu 10% de desconto.
Desconto: - R$ 300,00.

Frete normal grátis!
Sua compra atingiu R$ 500,00 ou mais.
Economia no frete: R$ 30,00.
```

CT08 — SENAI20 abaixo do valor mínimo

Entrada

- Produto: Mouse
- Quantidade: 5
- Subtotal: R$ 400,00
- Cupom: SENAI20

Resultado

O desconto não deve ser aplicado.

O sistema deve informar:

```
Cupom SENAI20 não aplicado.

O valor da sua compra é R$ 400,00.

Para receber 20% de desconto,
a compra precisa ser de no mínimo R$ 1.000,00.
```

```
Faltam R$ 600,00 para atingir o valor mínimo.

Desconto aplicado: R$ 0,00.
```

Essa situação representa uma **regra do sistema e não um erro**.

CT09 — SENAI20 atingindo o valor mínimo

Entrada

- Produto: Notebook
- Quantidade: 1
- Subtotal: R$ 3.000,00
- Cupom: SENAI20
- Frete: normal

Cálculo

Desconto:

```
R$ 3.000,00 × 20% = R$ 600,00
```

Frete:

```
R$ 0,00
```

Total inicial:

```
R$ 3.000,00 - R$ 600,00
= R$ 2.400,00
```

Como o total não ultrapassa R$ 3.000,00, o desconto de alto valor não é aplicado.

Resultado

```
Subtotal: R$ 3.000,00

Cupom SENAI20 aplicado!
Desconto de 20%: - R$ 600,00.

O desconto foi aplicado porque a compra
atingiu o valor mínimo de R$ 1.000,00.
```

```
Frete normal grátis!
Sua compra atingiu R$ 500,00 ou mais.

Total: R$ 2.400,00
```

8. ANÁLISE DOS RESULTADOS

Após a execução dos testes, foi possível verificar diferentes caminhos de execução do sistema.

No primeiro caso, a condição de quantidade não bloqueava o valor zero. O problema foi identificado através da análise do operador utilizado na condição. A alteração de `< 0` para `< 1` permitiu validar corretamente a quantidade mínima.

No segundo caso, a condição do estoque utilizava o operador >=, impedindo a compra exatamente na quantidade disponível. A alteração para > permitiu que o limite do estoque fosse utilizado corretamente.

No terceiro caso, foi analisada a regra de frete normal grátis. A regra de negócio foi mantida, pois compras a partir de R$ 500,00 possuem frete normal gratuito. O problema identificado foi a falta de comunicação clara ao cliente sobre o motivo do frete estar zerado. Após a correção, o sistema informa que o frete grátis foi concedido porque a compra atingiu o valor mínimo.

No quarto caso, foi analisado o desconto de 5% para compras com mais de 5 unidades. Essa regra também foi mantida. O problema era que o sistema aplicava o desconto sem informar claramente ao cliente o motivo e o valor descontado.

No quinto caso, foi analisado o desconto de 5% para pedidos acima de R$ 3.000,00. A regra foi mantida e a comunicação foi melhorada para informar ao cliente quando o benefício fosse aplicado.

No sexto caso, foram analisadas combinações entre cupom, frete e descontos adicionais. O principal problema encontrado foi a falta de detalhamento do resultado final. Após a correção, cada benefício passou a ser apresentado separadamente.

Também foram testadas as regras corretas dos cupons.

O cupom `SENAI10` aplica 10% de desconto.

O cupom `SENAI20` aplica 20% quando o subtotal é de pelo menos R$ 1.000,00.

Quando o valor mínimo do `SENAI20` não é atingido, o desconto não é aplicado e o sistema informa ao cliente quanto falta para atingir o valor mínimo.

O frete normal também foi testado. Quando o subtotal é de pelo menos R$ 500,00, o frete é gratuito e o sistema informa essa condição ao cliente.

Dessa forma, as regras comerciais continuam funcionando, mas agora são apresentadas de maneira mais clara.

9. COMPARAÇÃO ENTRE O COMPORTAMENTO ANTERIOR E APÓS A CORREÇÃO

Situação	Antes da correção	Após a correção
Quantidade 0	Era aceita	É bloqueada
Quantidade igual ao estoque	Era bloqueada	É permitida
Frete normal abaixo de R$ 500	R$ 30,00	R$ 30,00 informado ao cliente
Frete normal a partir de R$ 500	Ficava grátis, mas sem explicação	Continua grátis e o motivo é informado
Mais de 5 unidades	Aplicava 5% sem detalhamento	Aplica 5% e informa o motivo e valor
Total acima de R$ 3.000	Aplicava 5% sem detalhamento	Aplica 5% e informa o motivo e valor
SENAI10	Aplicava 10%	Aplica 10% e identifica o desconto
SENAI20 abaixo de R$ 1.000	Não aplicava	Não aplica e informa o valor mínimo
SENAI20 a partir de R$ 1.000	Aplicava 20%	Aplica 20% e identifica o desconto
Combinação de benefícios	Resultado pouco detalhado	Cada benefício é apresentado separadamente
Resultado final	Mostrava principalmente valores	Explica como cada valor foi calculado

10. CÓDIGO CORRIGIDO

Validação da quantidade

```
if (qtd < 1) {
   resultado.innerHTML = `
      <p><strong>Quantidade inválida.</strong></p>
      <p>A quantidade deve ser maior que zero.</p>
   `;
   return;
}
```

Validação do estoque

```
if (qtd > estoque[produtoSelecionado]) {
   resultado.innerHTML = `
      <p><strong>Quantidade indisponível em estoque.</strong></p>
      <p>Estoque disponível: ${estoque[produtoSelecionado]}
unidade(s).</p>
   `;
   return;
}
```

Cálculo dos cupons

```
function calcularDesconto(subtotal, codigo) {
   if (codigo === "SENAI10") {
      return subtotal * 0.10;
   }

   if (codigo === "SENAI20" && subtotal >= 1000) {
      return subtotal * 0.20;
   }

   return 0;
}
```

Cálculo do frete

```
function calcularFrete(tipo, subtotal) {
   if (tipo === "retirada") {
      return 0;
```

```
   }

   if (tipo === "expresso") {
      return 60;
   }

   if (subtotal >= 500) {
      return 0;
   }

   return 30;
}
```

A regra de frete grátis foi mantida.

Desconto por quantidade

```
if (qtd > 5) {
   descontoQuantidade = subtotal * 0.05;
   total = total - descontoQuantidade;
}
```

A regra foi mantida e passou a ser apresentada ao cliente.

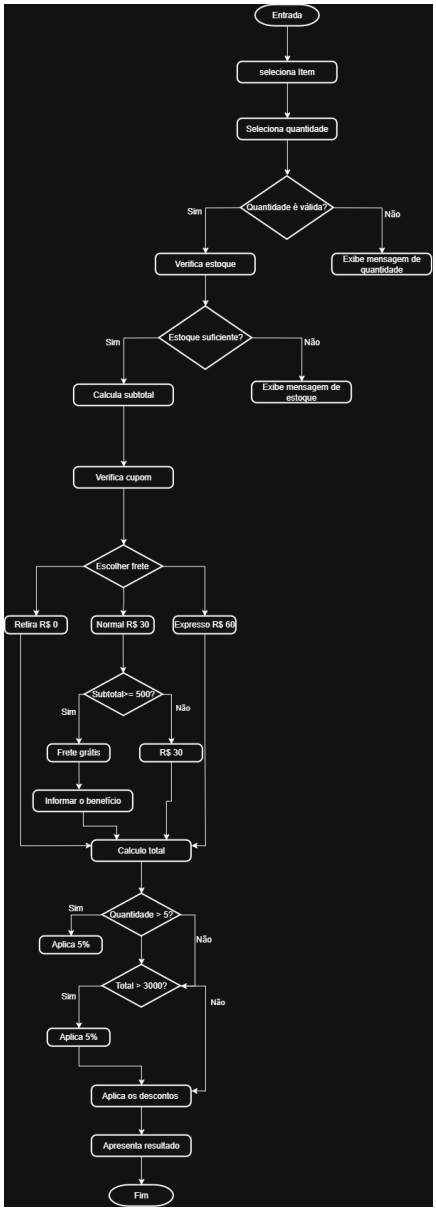
Desconto para pedido acima de R$ 3.000,00

```
if (total > 3000) {
   descontoAltoValor = total * 0.05;
   total = total - descontoAltoValor;
}
```

A regra também foi mantida e passou a ser apresentada ao cliente.

11. FLUXO FINAL DO SISTEMA

Após as correções, o fluxo do sistema ficou:



12. CONCLUSÃO

A realização do teste de caixa branca permitiu analisar a estrutura interna do sistema de pedidos da Loja SENAI, observando as condições, operadores, decisões e diferentes caminhos de execução presentes no código.

Através dos casos de teste, foram identificados seis problemas relacionados principalmente à validação de quantidade, controle do estoque e comunicação das regras de frete e descontos ao cliente.

A validação da quantidade foi corrigida para impedir valores menores que 1, enquanto a validação do estoque foi ajustada para permitir a compra exatamente na quantidade disponível.

As regras comerciais de desconto e frete foram mantidas. O frete normal continua sendo gratuito para compras a partir de R$ 500,00. O desconto por quantidade continua sendo aplicado para compras com mais de 5 unidades. O desconto para pedidos acima de R$ 3.000,00 também continua sendo aplicado.

Os cupons `SENAI10` e `SENAI20` também foram mantidos. O `SENAI10` concede 10% de desconto e o `SENAI20` concede 20% quando o subtotal da compra é de pelo menos R$ 1.000,00.

A principal melhoria realizada nessas regras foi a comunicação com o cliente. Quando um benefício é aplicado, o sistema agora identifica qual regra foi utilizada, qual foi o valor do desconto ou da economia e, quando necessário, explica o motivo pelo qual o benefício foi concedido.

No caso do `SENAI20`, quando a compra não atinge R$ 1.000,00, o sistema informa que o desconto não foi aplicado e mostra quanto falta para atingir o valor mínimo.

Da mesma forma, quando o frete normal é gratuito, o sistema informa que isso ocorreu porque o pedido atingiu R$ 500,00.

A atividade demonstrou a importância do teste de caixa bran ca para analisar não somente os valores finais, mas também os caminhos percorridos pelo programa, as condições executadas e a forma como cada regra interfere no resultado.

Após as correções, o sistema mantém suas regras de negócio e apresenta ao cliente informações mais claras sobre os descontos, benefícios, frete e valor final do pedido.